

TP_02 Compteur

1. L'horloge du système est fixée à 100MHz. Combien de période faut-il compter pour attendre 2 secondes ? Combien de bits faut-il au minimum pour représenter cette valeur ?

On sait que : $T(\text{période}) = \frac{1}{f(\text{fréquence})}$ soit $T = 1/100\text{MHz}$

$$T = \frac{1}{100 \cdot 10^6} = 10^{-8} = 10 \text{ ns}$$

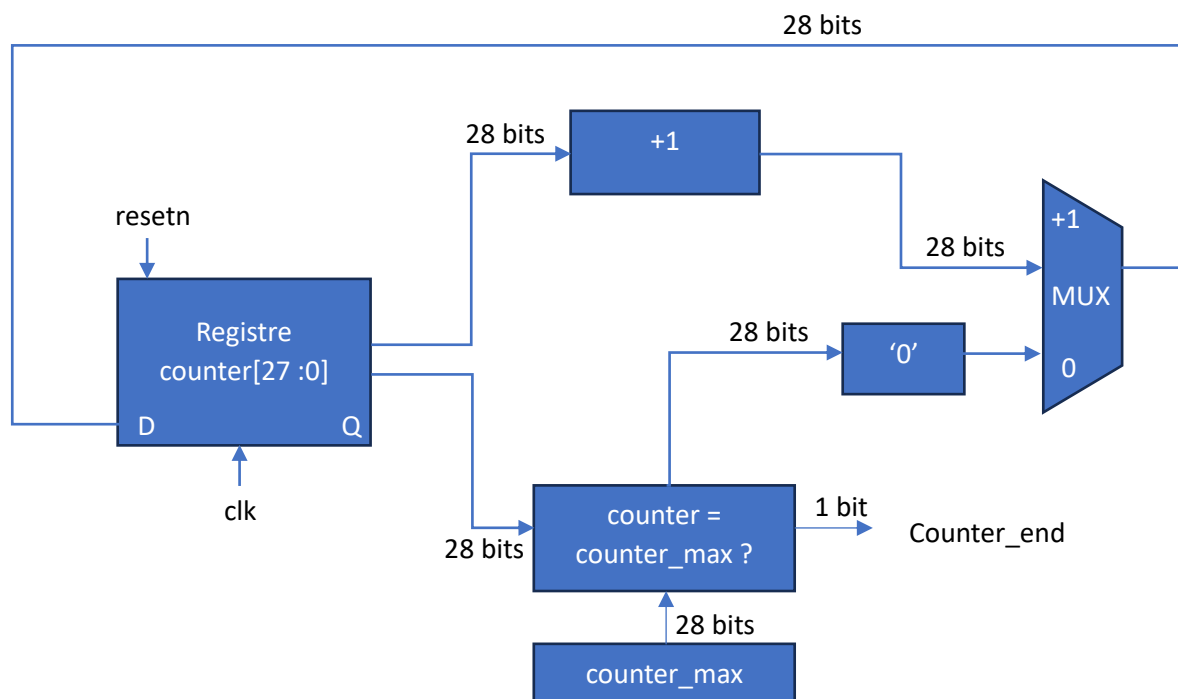
Pour avoir le nombre de coup d'horloge en 2 seconde :

$$\frac{2}{10^{-8}} = 200\,000\,000 \text{ périodes}$$

Et pour savoir le nombre de bits minimum pour représenter cette valeur il faut trouver une puissance de 2 supérieure à 200 000 000.

Soit : $2^{28} = 268\,435\,456$ donc 28 bits.

2. Dessinez le schéma RTL de ce compteur. Si le compteur atteint la valeur calculée précédemment, un signal end_counter passe à 1, sinon end_counter vaut 0. N'oubliez pas de mettre sur chaque signal son nombre de bits. Commencez par réaliser une boucle d'incrémentation : +1 à chaque coup d'horloge.
3. Ajoutez une condition pour que le compteur soit remis à 0 lorsqu'il a atteint la valeur souhaitée



4. Listez les signaux d'entrée, de sortie et les signaux internes de votre architecture

Entrées : clk, resetn

Sortie : counter_end

Signal interne : counter

5. Ecrivez à présent le compteur en VHDL en suivant le schéma RTL, faites attention de bien faire correspondre les noms des signaux de votre code VHDL avec ceux de votre schéma RTL.

(Pièce jointe)

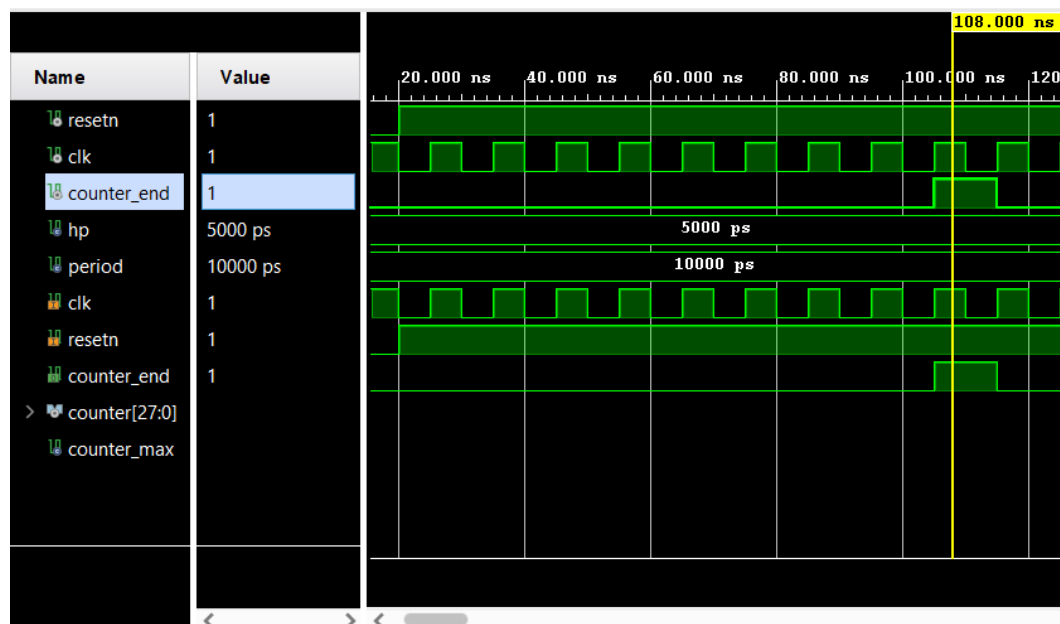
6. Ecrivez un fichier de testbench pour tester votre design.

(Pièce jointe)

7. Lancez une simulation. Que devez-vous observer sur votre chronogramme pour vérifier que votre design est valide ?

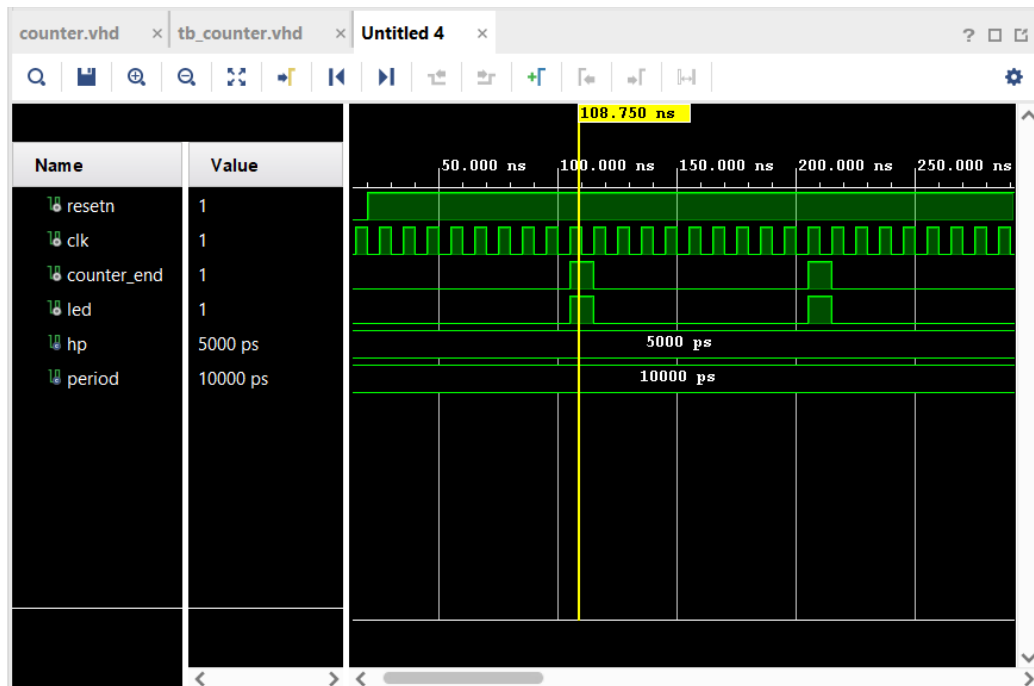
Pour vérifier que notre simulation est valide, il faudrait que le counter_end passe à 1 lorsque le compteur atteint sa valeur maximale.

Ici j'ai pris 9 comme valeur maximale afin de ne pas surcharger la simulation et compter jusqu'à 199 999 999.



On voit bien le counter_end égal à 1 après que le counter soit passé par les 10 valeurs.

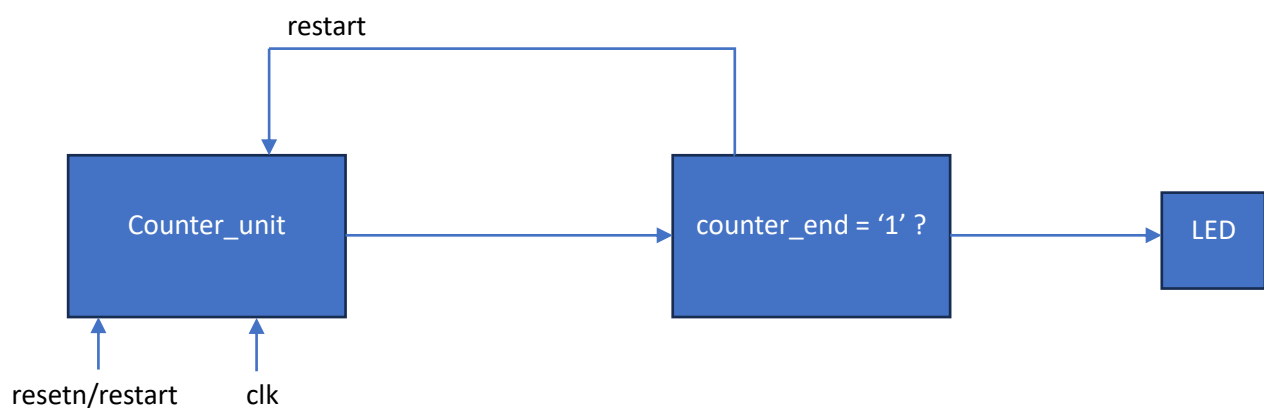
8. Associez une LED avec le signal de teste d'arrêt du compteur. Pour cela, il faudra ajouter une sortie et la relier à une broche d'une LED dans le fichier de contrainte (.xdc). La LED sera alors allumée pendant seulement un coup d'horloge.



On remarque que la LED passe à 1 au même moment que counter_end.

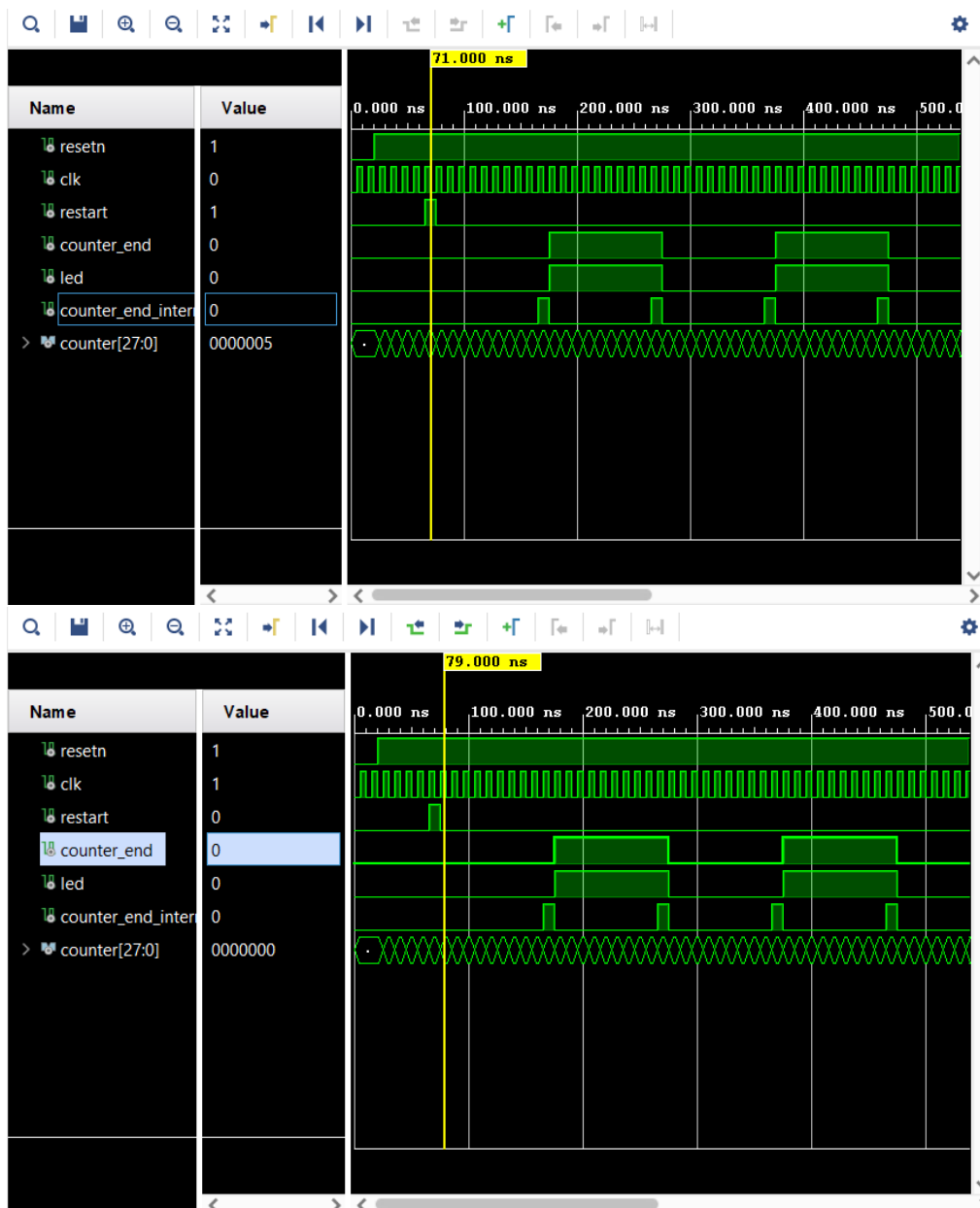
9. Modifiez le schéma RTL du compteur pour ajouter une remise à 0 lorsqu'un signal restart est à 1. Ajoutez la logique nécessaire pour que la LED clignote telle que : allumée 2s, éteinte 2s.

On prend le counter_unit comme base, puisqu'il à une période de 2 secondes, j'ajoute une entrée restart qui, comme resetn, remet le compteur à 0 lorsqu'elle passe à '1'. La LED bascule automatiquement à chaque fois que counter_end est égal à '1', soit :

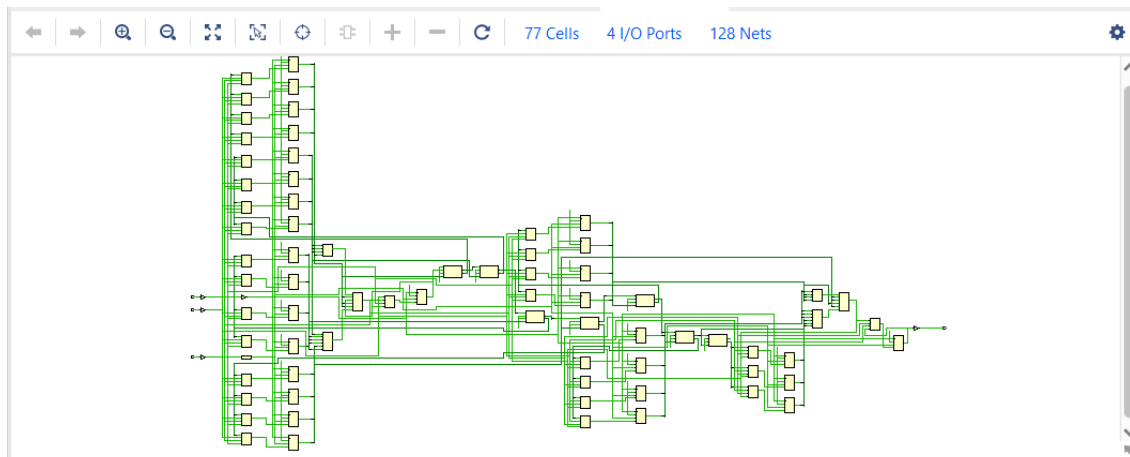


10. Faites les mises à jour nécessaires sur le code VHDL pour correspondre au nouveau schéma. Le signal restart sera une entrée du design.
(pièce jointe)
11. Associez la nouvelle entrée restart à un bouton.
(pièce jointe)
12. Mettez à jour votre testbench puis vérifiez votre design avec une simulation. Quels sont les signaux que vous devez observer ?

J'ai ajouté les signaux counter, restart et led, on peut voir que suite au restart le compteur redémarre à 0 et que la led oscille entre 0 et 1 entre chaque série de compteur, donc tout est bon.



13. Exécutez la synthèse puis ouvrez la schématique. Identifiez sur la schématique les différents éléments de votre architecture RTL.



On peut voir l'ensemble des registres sur la gauche du schéma, au centre il y a le compteur ainsi que le comparateur avec counter_max et les multiplexeurs. Et sur la droite on retrouve la sortie counter_end

14. Ouvrez le rapport de synthèse et relevez les ressources utilisées. Comparez vos résultats avec les résultats attendu selon votre architecture RTL.

La synthèse de notre architecture utilise 29 bascules (Slice Registers), ce qui correspond exactement aux 28 bits du registre de comptage plus le bit mémorisant l'état de la LED. Les 22 LUTs semblent assez logiques. Enfin, les 3 broches d'entrée et la broche de sortie correspondent aux 4 ports de notre entité (clk, resetn, restart, counter_end). Tout paraît assez cohérent.

15. Ouvrez le Set Up Debug. Placez des sondes sur les signaux à observer que vous avez défini à la question 12.

J'ai placé des sondes sur counter, counter_end, restart et reset_btn.

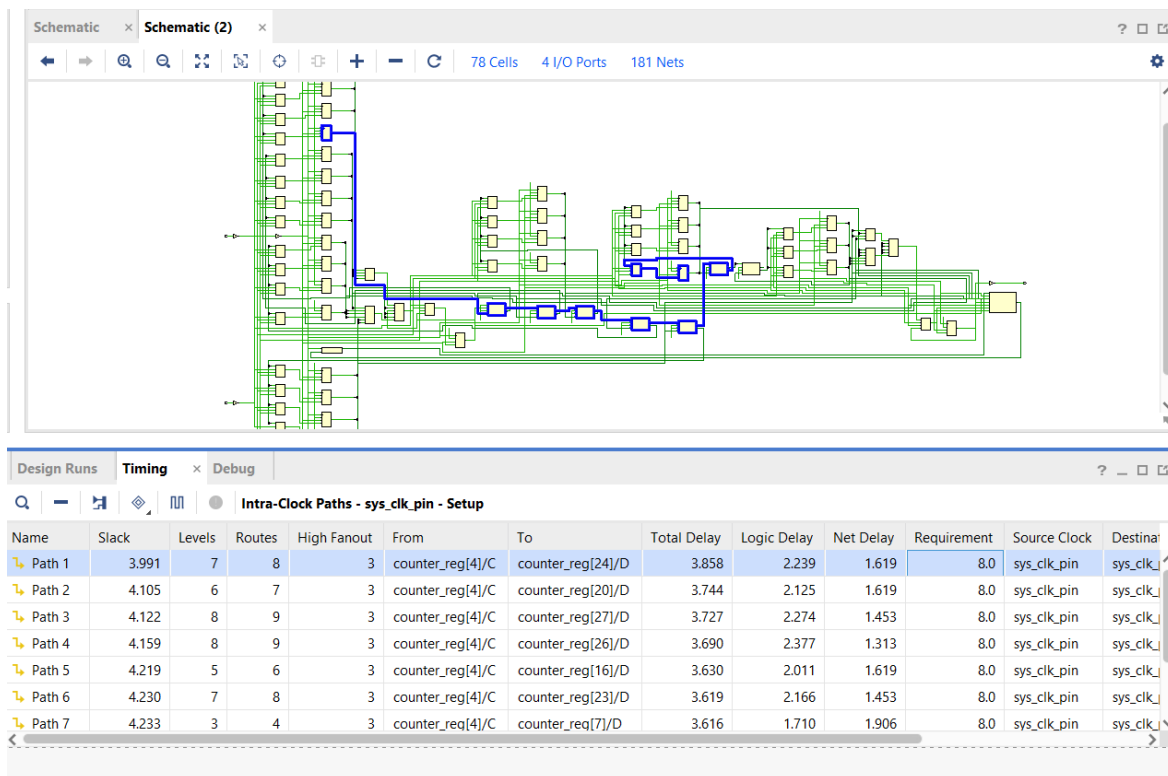
16. Lancez l'implémentation puis étudiez le rapport de timing (vérifiez les violations de set up et de hold et identifiez le chemin critique).

Après le lancement de l'implémentation, j'ai pu étudier le rapport de timing tout semble cohérent, comme le précise le rapport « All user specified timing constraints are met ».

Le rapport indique un WNS de 3,991 ns et un WHS de 0,323 ns. Le premier chemin va de counter_reg[4] à counter_reg[24].

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3,991 ns	Worst Hold Slack (WHS): 0,323 ns	Worst Pulse Width Slack (WPWS): 3,500 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 29	Total Number of Endpoints: 29	Total Number of Endpoints: 30
All user specified timing constraints are met.		



Par contre j'observe que la période requise pour chaque « path » est de 8.0 ns et non 10 ns comme calculé à la question 1.

C'est peut-être pour cela que l'intervalle entre les clignotements de la LED me paraisse plus court que 2 secondes (à revoir).

17. Générez le bitstream pour observer le système sur carte et le résultat de la ILA.

Après avoir généré le Bitstream, je peux observer que la LED verte sur la CORA Z7 clignote avec un intervalle de plus ou moins 2 secondes.

De plus lorsque j'appuie sur un des 2 boutons programmés pour le restart et le resetn, la LED s'éteint instantanément et redémarre le compteur. Tout a l'air de fonctionner correctement !